Kubernetes Cheatsheet

Essential commands for container orchestration and cluster management

This comprehensive cheatsheet covers Kubernetes fundamentals including cluster setup, kubectl commands, and best practices, ideal for both beginners and experienced developers managing containerized applications.

Basic Commands	Pod Management	Deployments
Essential kubectl operations	Create and manage pods	Application deployment & scaling

Services & Networking	Configuration
Expose and connect applications	ConfigMaps and Secrets

Installation & Setup

Install kubectl

Install the Kubernetes command line tool.

```
# macOS with Homebrew
brew install kubectl

# Linux official binary
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd
64/kubectl"
chmod +x kubectl

# Windows with Chocolatey
choco install kubernetes-cli
```

Verify Installation

Check kubectl version and cluster connection.

```
# Check kubectl version
kubectl version --client

# Check both client and server versions
kubectl version

# Get cluster information
kubectl cluster-info

# Get cluster configuration
kubectl config view

# Show available API resources
kubectl api-resources
```

Configure kubectl

Set up kubectl to connect to your cluster.

```
# Set current context
kubectl config use-context <context-name>

# Set default namespace
kubectl config set-context --current --namespace=my
-namespace

# List contexts
kubectl config get-contexts

# Delete context
kubectl config delete-context <context-name>
```

Minikube Setup

Quick local Kubernetes cluster for development.

```
# Start minikube
minikube start

# Check status
minikube status

# Access dashboard
minikube dashboard

# Stop cluster
minikube stop

# Delete cluster
minikube delete
```

Basic Commands & Cluster Info

Fundamental operations to understand and interact with your Kubernetes cluster.

Cluster Information	kubectl
Display essential cluster details and service endpoints	View and manage cluster nodes.

```
# Cluster information
kubectl cluster-info

# Detailed cluster information
kubectl cluster-info dump

# Check API server health
kubectl get --raw /healthz

# Get cluster version
kubectl version --short
```

```
# List all nodes
kubectl get nodes

# Detailed node information
kubectl get nodes -o wide

# Describe specific node
kubectl describe node <node-name>

# Get node resource usage
kubectl top nodes
```

Pod Management

Create & Run Pods: `kubectl run` / `kubectl create`

Launch containers and manage their lifecycle.

```
# Run a simple pod
kubectl run nginx --image=nginx

# Create pod from YAML file
kubectl create -f pod.yaml

# Run pod and connect interactively
kubectl run busybox --image=busybox --rm -it --
/bin/sh

# Create pod with port
kubectl run nginx --image=nginx --port=80 --
expose
```

View Pod Information: `kubectl get pods`

Monitor pod status and gather information.

```
# List all pods in default namespace
kubectl get pods

# List pods with more details
kubectl get pods -o wide

# Watch pods in all namespaces
kubectl get pods -A --watch

# Watch pod status changes
kubectl get pods --watch

# Pod Details: kubectl describe pod
kubectl describe pod <pod-name>

# Describe pod in specific namespace
kubectl describe pod -n <namespace>

# List pod logs
kubectl logs <pod-name>

# Follow logs in real-time
kubectl logs -f <pod-name>

# Pod Operations: kubectl exec / kubectl
delete
Execute commands & pod management.

# Execute command in pod
kubectl exec -it -- /bin/bash

# Execute command in specific container
kubectl exec -it -c -- sh

# Delete pod
kubectl delete pod

# Force delete a pod
kubectl delete pod --grace-period=0 --force
```

Deployments & ReplicaSets

Create Deployments: `kubectl create deployment`

Deploy and manage application deployments.

```
# Create deployment
kubectl create deployment nginx --image=nginx

# Create deployment in namespace
kubectl create deployment webapp --image=myapp -
-replicas=3

# Create from YAML file
kubectl create -f deployment.yaml

# Expose deployment as service
kubectl expose deployment nginx --port=80 --
type=LoadBalancer

# Scaling: kubectl scale
Adjust the number of running replicas.

# Scale deployment
kubectl scale deployment nginx --replicas=5

# Scale ReplicaSet
kubectl scale rs --replicas=3

# Auto-scale deployment
kubectl autoscale deployment nginx --min=2 --max=10
--cpu-percent=80

# Rolling Updates: kubectl rollout
Manage deployment updates and rollbacks.

# Check rollout status
kubectl rollout status deployment/nginx

# View rollout history
kubectl rollout history deployment/nginx

# Rollback to previous version
kubectl rollout undo deployment/nginx

# Rollback to specific revision
kubectl rollout undo deployment/nginx --to-revision=2
```

Services & Networking

Expose Services: `kubectl expose`

Make applications accessible via network services.

```
# Expose deployment as ClusterIP service
kubectl expose deployment nginx --port=80

# Expose as NodePort service
kubectl expose deployment nginx --port=80
--type=NodePort

# Expose as LoadBalancer
kubectl expose deployment nginx --port=80
--type=LoadBalancer

# Expose service
kubectl expose service nginx --port=80

# Port Forwarding: kubectl port-forward
Access applications locally for testing and debugging.

# Forward port pod to local machine
kubectl port-forward pod/nginx 8080:80

# Forward service port
kubectl port-forward service/nginx 8080:80

# Forward deployment port
kubectl port-forward deployment/nginx 8080:80

# Forward multiple ports
kubectl port-forward pod/nginx 8080:80 8443:443

# Ingress Management
Manage external access to services via HTTP/HTTPS routes.

# List ingress resources
kubectl get ingress

# Describe ingress
kubectl describe ingress

# Create ingress from YAML
kubectl apply -f ingress.yaml

# Service Types: kubectl get
Manage different types of services with various properties.

# List all services
kubectl get services

# List services with more details
kubectl get services -o wide

# Describe specific service
kubectl describe service

# Get service endpoints
kubectl get endpoints

# Delete service
kubectl delete service <service-name>
```

ConfigMaps & Secrets

Manage configuration data and sensitive information separately from application code.

ConfigMaps: `kubectl create configmap`

Store non-confidential configuration data in key-value pairs.

```
# Create ConfigMap from literal values
kubectl create configmap app-config --from-
literal=debug=true --from-literal=log-level=info

# Create from file
kubectl create configmap app-config --from-file=
./config.properties

# Create from directory
kubectl create configmap app-config --from-file=config/

# Create available API resources
kubectl get configmaps

# Delete ConfigMap
kubectl delete configmap app-config

**Secrets:** `kubectl create secret`

Store sensitive data such as passwords and API keys.

# Create generic secret
kubectl create secret generic db-secret --from
-literal=username=admin --from-literal=
password=secret123

# Create secret from file
kubectl create secret generic api-secret --from-file=./
api.key --from-file=./api.crt

# Create docker registry secret
kubectl create secret docker-registry my-registry
--docker-server=myregistry.com --docker-username=user
--docker-password=pass

**Secret Management**

View and manage secrets securely.

# List secrets
kubectl get secrets

# Describe secret (values are hidden)
kubectl describe secret db-secret

# Decode secret values
kubectl get secret db-secret -o
jsonpath='{.data.password}' | base64 -d

# Delete secret
kubectl delete secret db-secret
```

Storage & Volumes

Manage persistent data and configure storage volumes for containerized applications.

Persistent Volumes: `kubectl get pv`

Manage cluster-wide storage resources.

```
# List persistent volumes
kubectl get pv

# Describe persistent volume
kubectl describe pv

# Create PV from YAML
kubectl apply -f persistent-volume.yaml

# Delete persistent volume
kubectl delete pv

**Persistent Volume Claims:** `kubectl get pvc`

Request storage resources for pods.

# List PVCs
kubectl get pvc

# Describe PVC
kubectl describe pvc

# Create PVC from YAML
kubectl apply -f pvc.yaml

# Delete PVC
kubectl delete pvc

**Storage Classes:** `kubectl get storageclass`

Define different types of storage with various properties.

# List storage classes
kubectl get storageclass

# Describe storage class
kubectl describe storageclass

# Set default storage class
kubectl patch storageclass fast -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default
-class":"true"}}}'

**Volume Operations**

Work with different volume types in your pods.

# Check volume mounts in pod
kubectl describe pod | grep -A 5 "Mounts:"

# List volumes in pod
kubectl get pod -o jsonpath='{.spec.volumes}'

**Resource Usage:** `kubectl top`

Monitor resource consumption for pods and nodes.

# View node resource usage
kubectl top nodes

# View pod resource usage
kubectl top pods

# View pod resource usage in namespace
kubectl top pods -n

# Sort pods by CPU usage
kubectl top pods --sort-by=cpu

**Interactive Debugging:** `kubectl exec` / `kubectl debug`

Access running containers for hands-on troubleshooting.

# Execute interactive shell
kubectl exec -it -- /bin/bash

# Debug with ephemeral container (k8s 1.23+)
kubectl debug -it --image=busybox

# Copy files from pod
kubectl cp /local/file pod:/remote/path

# Copy files to pod
kubectl cp /local/file /path/to/destination
```

Resource Management

Apply, update, and manage Kubernetes resources using YAML manifests.

Apply Resources: `kubectl apply`

Create or update resources using declarative configuration files.

```
# Apply single file
kubectl apply -f deployment.yaml

# Apply multiple files
kubectl apply -f deployment.yaml -f service.yaml

# Apply entire directory
kubectl apply -f ./manifests/

# Apply from URL
kubectl apply -f https://example.com/manifest.yaml

# Show what would be applied (dry run)
kubectl apply -f deployment.yaml --dry-run=client -o yaml

**Resource Operations:** `kubectl get` / `kubectl delete`

Get, inspect, and remove Kubernetes resources.

# List all resources in namespace
kubectl get all

# List resources with custom columns
kubectl get pods -o custom
-columns=NAME:.metadata.name,STATUS:.status.phase

# Get resources as JSON/YAML
kubectl get deployment nginx -o yaml
kubectl get pod -o jsonpath='{.status.phase}'

# Delete resources
kubectl delete -f deployment.yaml
kubectl delete pods,services -l app=nginx

**Resource Editing:** `kubectl edit` / `kubectl patch`

Modify existing resources directly.

# Edit resource interactively
kubectl edit deployment nginx

# Patch resources with strategic merge
kubectl patch deployment nginx -p '{"spec":
{"replicas":3}}'

# Patch using JSON format
kubectl patch pod nginx --type=json -p '[{"op":
"replace","path":"/spec/containers/0/image",
"value":"nginx:1.21"}]'

# Replace resource entirely
kubectl replace -f updated-deployment.yaml

**Resource Validation:** `kubectl diff` / `kubectl explain`

Compare configurations and understand resource schemas.

# Show differences between applied
kubectl diff -f deployment.yaml

# Explain resource structure
kubectl explain pod.spec.containers

# Explain with recursive
kubectl explain deployment --recursive

# Validate resource before applying
kubectl apply -f deployment.yaml --validate=true

## Advanced Operations

**Node Management:** `kubectl cordon` / `kubectl drain`

Manage node availability for maintenance and updates.

```
Mark node as unschedulable
kubectl cordon node-1

Mark node as schedulable
kubectl uncordon node-1

Drain node for maintenance
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data

Add taint to node
kubectl taint nodes node1 key=value:NoSchedule

Remove taint from node
kubectl taint nodes node1 key=value:NoSchedule-

Advanced Debugging: `kubectl label` / `kubectl annotate`

Add metadata for organization and advanced debugging.

Add label to resource
kubectl label pod environment=production

Remove label from resource
kubectl label pod environment-

Add annotation to resource
kubectl annotate pod description="Production web
server"

Remove annotation
kubectl annotate pod description-

Proxy & Authentication: `kubectl proxy` / `kubectl auth`

Secure cluster access and API exploration.

Start proxy to Kubernetes API
kubectl proxy --port=8080

Check if user can perform action
kubectl auth can-i create pods

Impersonate another user
kubectl get pods --as=jane --as-group=dev-team

Impersonate service account
kubectl get pods --as=system:serviceaccount:default:my-sa

Generate kubeconfig for service account
kubectl get pod -n kube-system -l k8s-app=kube-dns
-o jsonpath='{.items[0].metadata.creationTimestamp}'

Utility Commands

Additional kubectl commands for Kubernetes operations.

Wait for condition
kubectl wait --for=condition=Ready pod/nginx --timeout=300s

Run temporary pod for testing
kubectl run test-pod --rm -i --image=busybox --
/bin/sh

Generate resource YAML without creating
kubectl create deployment nginx --image=nginx --dry
-run=client -o yaml

Port-forward to multiple ports
kubectl get pods -n kube-system -l k8s-app=kube-dns

Performance & Monitoring

Monitor cluster health and optimize resource utilization.

Resource Metrics: `kubectl top`

View real-time resource usage.

```
# Node resource usage
kubectl top nodes --sort-by=cpu
kubectl top nodes --sort-by=memory

# Pod resource usage
kubectl top pods --sort-by=cpu
kubectl top pods --sort-by=memory

# Container resource usage
kubectl top pods --containers

# Historical resource usage (requires metrics-server)
kubectl top pod --heapster

**Health Checks & Status**

Monitor cluster components and diagnose issues.

# Check deployment rollout status
kubectl rollout status deployment/

# Check pod readiness
kubectl get pod -o jsonpath='{.status.phase}{\n}{
.status.conditions[?(@.type==Ready)].status}'

# Monitor resource quotas
kubectl get resourcequota
kubectl describe resourcequota

# Check cluster component status
kubectl get componentstatuses

**Performance Optimization**

Commands to help optimize cluster performance.

# Node resource requests and limits
kubectl describe node | grep -A5 "Allocated resources:"

# Check pod disruption budgets
kubectl get pdb

# View horizontal pod autoscalers
kubectl get hpa

# Check resource policies
kubectl get limitranges

**Backup & Recovery**

Essential commands for cluster backup and disaster recovery.

# Backup all resources in namespace
kubectl get all -n prod > k-backup.txt

# Export secret for backup
kubectl get deployment -n prod > deployment
backup.yaml

# List all resources for backup
kubectl api-resources --verbs=list --namespaced -o name
| xargs -n 1 kubectl get --show-kind --ignore-not-found -n

## Configuration & Context Management

Manage multiple clusters and configure kubectl for different environments.

**Context Management**

Switch between different Kubernetes clusters and contexts.

```
# View current context
kubectl config current-context

# List all contexts
kubectl config get-contexts

# Switch context
kubectl config use-context

# Create new context
kubectl config set-context dev-
cluster --namespace=development --
user=dev-user
export KUBECONFIG=~/.kube/config:~/.kube/prod-
config:~/.kube/staging-config
kubectl config view --merge --flatten >
~/.kube/merged-config

**Kubeconfig Management**

Organize and manage multiple cluster configurations.

# View merged kubeconfig
kubectl config view

# Use specific kubeconfig
export KUBECONFIG=~/.kube/prod-config
kubectl get nodes

# Backup kubeconfig
cp ~/.kube/config ~/.kube/config.backup

# Validate kubeconfig
kubectl config view --validate

**Default Settings**

Set default namespaces and preferences for faster operations.

# Set default namespace for current context
kubectl config set-context --
current --namespace=dev

# Set different output format as default
kubectl config set-context --
current --output=wide

# View configuration details
kubectl config view --minify
kubectl config view --raw --flatten
kubectl config view --raw
```

Authentication: This cheatsheet covers kubectl commands and Kubernetes operations for managing containerized applications in production environments. For the most current information, refer to the **official Kubernetes documentation**.
```